Notebook de BolitosFC

Camilo Sodo, Gaspar Hilman, Juan Ignacio Varisco

2026

Índice general

1. Generalidades **1**
1.1. Plantilla 1
1.2. Makefile 1
1.3. Busqueda Binaria 2

2. Grafos **3**
2.1. Recorridos 3
2.2. Diametro en Arboles 4
2.3. Dijkstra 5
2.4. Ascenso Binario 5
2.5. Primer Ancestro Común 6
2.6. Componentes Furtemente Conexas 7
2.7. Ordenamiento Topologico 8

3. Teoria de Números **10**
3.1. Minimo Comun Multiplo/Divisor Comun Menor 10
3.2. Criba de Eratóstenes 10
3.3. Exponenciación Binaria 11

4. Estructuras de Datos **12**
4.1. Segment Tree 12
4.2. Lazy Segment Tree 12
4.3. Trie 14
4.4. Union-Find 15

5. Códigos de Ejemplo **17**
5.1. Codigo 1 17
5.2. Codigo 2 18
5.3. Codigo 3 20

Generalidades

1.1. Plantilla

Plantilla de código para competencias.

```cpp
#include <bits/stdc++.h>
#define ll long long
#define dd long double
#define ull unsigned long long
#define forr(i, h) for (ll i = 0; i < h; i++)
#define forrr(i, d, h) for (ll i = d; i < h; i++)
#define techo(x, k) ((x + k - 1) / k)
#define endl '\n'
#define initArr(arr, largo, contenido)

    \
    for (int i = 0; i < largo; i++) arr[i] = contenido;
using namespace std;
using Par = pair<ll, ll>;
using GrafoPesado = vector<vector<pair<ll, ll>>>;
using Grafo = vector<vector<ll>>;
using Digrafo = vector<vector<pair<ll, bool>>>;
using Arbol = vector<vector<ll>>;

int main() {
#ifdef LOCAL
    freopen(FILE_DIR, "r", stdin);
#endif
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    ll casos;
    while (cin >> casos) {
        while (casos--) {
        }
    }
}
```

1.2. Makefile

Makefile para compilar el codigo mas comodamente.

```
SHELL := /bin/bash
FILE_NAME := $(basename $(notdir $(FILE)))
DIR_NAME := $(dir $(FILE))
OUT_NAME := $(DIR_NAME)$(FILE_NAME).out
IN_NAME := $(DIR_NAME)$(FILE_NAME).in
```

```
6
7  run: compile
8    ./$(OUT_NAME)
9
10 debug: compile
11   gdb -x ./comandosGDB ./$(OUT_NAME)
12
13 compile: $(FILE)
14   g++ $(FILE) -DLOCAL -DFILE_DIR='"./$(IN_NAME)"' -g -fsanitize=address,
       undefined -o $(OUT_NAME) -std=c++17
15
16 clear:
17   rm *.out
18
19 fill:
20   mkdir $(DIR)
21   cat template.cpp | tee $(DIR)/{a..n}.cpp > /dev/null
22   touch $(DIR)/{a..n}.in
23
24 traer:
25   cp $(DIR)/.clang-format .clang-format
26   cp $(DIR)/makefile makefile
27   cp $(DIR)/comandosGDB comandosGDB
28   cp $(DIR)/template.cpp template.cpp
29
30 llevar:
31   cp .clang-format $(DIR)/.clang-format
32   cp makefile $(DIR)/makefile
33   cp comandosGDB $(DIR)/comandosGDB
34   cp template.cpp $(DIR)/template.cpp
```

1.3. Busqueda Binaria

Plantilla de una busqueda binaria basica.

```
1  #define ll long long
2  using namespace std;
3
4  /* Binary search de Gaspi */
5  bool func(ll x);
6
7  ll solve(ll min_val, ll max_val) {
8      ll L = min_val;
9      ll R = max_val;
10     ll ans = -1;
11     while (L <= R) {
12         ll mid = L + (R - L) / 2;
13         if (func(mid)) {
14             ans = mid;
15             R = mid - 1;
16         } else {
17             L = mid + 1;
18         }
19     }
20     return ans;
21 }
```

Grafos

2.1. Recorridos

Plantillas para recorridos BFS y DFS.

```cpp
#include <bits/stdc++.h>
#define ll long long
using namespace std;
using GrafoPesado = vector<vector<pair<ll, ll>>>;
using Grafo = vector<vector<ll>>;
using Digrafo = vector<vector<pair<ll, bool>>>;
using Arbol = vector<vector<ll>>;

void bfs(Grafo &g, ll raiz) {
    vector<bool> visitados(g.size(), false);
    queue<ll> q;
    q.push(raiz);
    while (!q.empty()) {
        ll actual = q.front();
        q.pop();
        visitados[actual] = true;
        for (ll vecino : g[actual]) {
            if (visitados[vecino]) continue;
            q.push(vecino);
        }
    }
}

void dfs(Grafo &g, ll raiz) {
    vector<bool> visitados(g.size(), false);
    stack<ll> p;
    p.push(raiz);
    while (!p.empty()) {
        ll actual = p.top();
        p.pop();
        visitados[actual] = true;
        for (ll vecino : g[actual]) {
            if (visitados[vecino]) continue;
            p.push(vecino);
        }
    }
}

void dfs_recursivo(Grafo &g, ll raiz, vector<bool> &visitado) {
    visitado[raiz] = true;
    for (ll vecino : g[raiz]) {
        if (visitado[vecino]) continue;
        dfs_recursivo(g, vecino, visitado);
```

```
    }
}
```

2.2. Diametro en Arboles

Calcula el camino mas largo de un árbol (su diametro).

```
#include <bits/stdc++.h>
#define ll long long
#define forr(i, h) for (ll i = 0; i < h; i++)
using namespace std;

/* Maxima distancia desde un vertice */

using Grafo = vector<vector<ll>>;
vector<ll> caminoDistancia;
ll maximaDistanciaDesde(Grafo &g, ll desde) {
    caminoDistancia = vector<ll>();
    vector<bool> encolado(g.size(), false);
    vector<ll> padres(g.size(), -1);
    queue<ll> q;
    ll lejano;
    q.push(desde);
    while (!q.empty()) {
        ll actual = q.front();
        q.pop();
        lejano = actual;
        for (ll vecino : g[actual]) {
            if (encolado[vecino]) continue;
            q.push(vecino);
            encolado[actual] = true;
        }
    }

    ll actual = lejano;
    while (actual != -1) {
        caminoDistancia.push_back(actual);
        actual = padres[actual];
    }
    reverse(caminoDistancia.begin(), caminoDistancia.end());
    return caminoDistancia.size();
}

/* Calculo de Diametro */

using Arbol = vector<vector<ll>>;
vector<ll> calcularDiametro(Arbol &g, ll nodo) {
    maximaDistanciaDesde(g, nodo);
    ll extremo = caminoDistancia.back();
    maximaDistanciaDesde(g, extremo);
    return caminoDistancia;
}

/* Centro del arbol (medio del diametro) (minimiza distancia maxima)*/

ll calcularCentro(vector<ll> diametro) { return diametro[diametro.size()
    / 2]; }
```

2.3. Dijkstra

Busca para todos los vertices la distancia minima desde un punto de partida dado.

```cpp
#include <bits/stdc++.h>
#define ll long long
using namespace std;

vector<ll> distanciasDijkstra;
using GrafoPesado = vector<vector<pair<ll, ll>>>;
using ElementoDijkstra = pair<ll, ll>;
// TODO: testear
void Dijkstra(GrafoPesado g, ll inicial) {
    distanciasDijkstra = vector<ll>(g.size(), 10e10);
    distanciasDijkstra[inicial] = 0;
    priority_queue<ElementoDijkstra, vector<ElementoDijkstra>, greater<
        ElementoDijkstra>> queue;
    queue.push({0, inicial});
    while (!queue.empty()) {
        auto [distanciaActual, actual] = queue.top();
        queue.pop();
        if (distanciasDijkstra[actual] < distanciaActual) continue;
        for (auto [vecino, largoArista] : g[actual]) {
            if (distanciasDijkstra[vecino] > largoArista +
                distanciaActual) {
                distanciasDijkstra[vecino] = largoArista +
                    distanciaActual;
                queue.push({distanciasDijkstra[vecino], vecino});
            }
        }
    }
}
```

2.4. Ascenso Binario

En arboles, permite encontrar un ancestro arbitrario de un vertice cualquiera rapidamente.

```cpp
#include <bits/stdc++.h>
#define ll long long
#define forr(i, h) for (ll i = 0; i < h; i++)
using namespace std;

using Arbol = vector<vector<ll>>;
vector<vector<ll>> ancestros; // La raiz es su propio ancestro
vector<ll> profundidad;
void calculaPrimerAncestro(Arbol &g, ll raiz) {
    stack<ll> p;
    p.push(raiz);
    ancestros[raiz].push_back(raiz);
    profundidad[raiz] = 0;
    while (!p.empty()) {
        auto act = p.top();
```

```
        p.pop();
        for (ll vecino : g[act]) {
            if (profundidad[vecino] == -1) {
                profundidad[vecino] = profundidad[act] + 1;
                ancestros[vecino].push_back(act);
                p.push(vecino);
            }
        }
    }
}

void calculaKesimoAncestro(Arbol &g, ll k) {
    forr(i, g.size()) {
        ll ancestro = ancestros[ancestros[i][k - 1]][k - 1]; // Cuidado
            si se cambia el ancestro de la raiz estandar
        ancestros[i].push_back(ancestro);
    }
}

void calcularBinaryLifting(Arbol &g, ll raiz) {
    ancestros = vector<vector<ll>>(g.size());
    profundidad = vector<ll>(g.size(), -1);
    calculaPrimerAncestro(g, raiz);
    for (ll cont = 2, k = 1; cont < g.size(); cont *= 2, k++) {
        calculaKesimoAncestro(g, k);
    }
}

ll enecimoAncestro(Arbol &g, ll nodo, unsigned ll n) {
    for (unsigned ll salto = 0; n != 0; n >>= 1, salto++) {
        if ((n & 1) != 0) {
            nodo = ancestros[nodo][salto];
        }
    }
    return nodo;
}
```

2.5. Primer Ancestro Común

En arboles, permite encontrar rapidamente el menor ancestro común (lowest common ancestor) entre dos vertices arbitrarios. Utiliza ascenso binario.

```
#include <bits/stdc++.h>
#define ll long long
#define forr(i, h) for (ll i = 0; i < h; i++)
using namespace std;

using Arbol = vector<vector<ll>>;
vector<vector<ll>> ancestrosLCA;
vector<ll> profundidadLCA;

// Utiliza Binary Lifting. Ver binaryLifting.cpp
void calcularBinaryLifting(Arbol &g, ll raiz); //
ll enecimoAncestro(Arbol &g, ll nodo, unsigned ll n);

void calcularLCA(Arbol &g, ll raiz) { calcularBinaryLifting(g, raiz); }
```

```
15
16 ll lowestCommonAncestor(Arbol &g, ll a, ll b) {
17     if (profundidadLCA[a] < profundidadLCA[b]) swap(a, b);
18
19     a = enecimoAncestro(g, a, profundidadLCA[a] - profundidadLCA[b]);
20
21     while (a != b) {
22         ll L = 0;
23         ll R = ancestrosLCA[b].size() - 1;
24         ll ans = 0;
25         while (L <= R) {
26             ll mid = L + (R - L) / 2;
27             if (ancestrosLCA[a][mid] != ancestrosLCA[b][mid]) {
28                 ans = mid;
29                 L = mid + 1;
30             } else {
31                 R = mid - 1;
32             }
33         }
34         a = ancestrosLCA[a][ans];
35         b = ancestrosLCA[b][ans];
36     }
37     return a;
38 }
```

2.6. Componentes Furtemente Conexas

Permite encontrar las componentes fuertemente conexas de un digrafo.

```
#include <bits/stdc++.h>
#define ll long long
#define forr(i, h) for (ll i = 0; i < h; i++)
#define endl '\n'
using namespace std;
using Digrafo = vector<vector<pair<ll, bool>>>;

vector<ll> componente;
vector<bool> visitado;
stack<ll> semiOrden;
#define CENTINELA -1
void visitarKosaraju(Digrafo &g, ll nodo) {
    if (visitado[nodo]) return;
    visitado[nodo] = true;
    for (auto [vecino, salida] : g[nodo]) {
        if (!salida) continue;
        visitarKosaraju(g, vecino);
    }
    semiOrden.push(nodo);
}

void asignarKosaraju(Digrafo &g, ll nodo, ll comp) {
    if (componente[nodo] != CENTINELA) return;
    componente[nodo] = comp;
    for (auto [vecino, salida] : g[nodo]) {
        if (salida) continue;
        asignarKosaraju(g, vecino, comp);
```

```
    }
}

// Determina componentes fuertemente conexas de un digrafo (aristas [vecino,
// salida?])
void algoritmoKosaraju(Digrafo &g) {
    componente = vector<ll>(g.size(), CENTINELA);
    visitado = vector<bool>(g.size(), false);
    forr(nodo, g.size()) { visitarKosaraju(g, nodo); }
    ll nroComponentes = 0;
    while (!semiOrden.empty()) {
        ll nodo = semiOrden.top();
        asignarKosaraju(g, nodo, nroComponentes);
        nroComponentes = max(nroComponentes, componente[nodo] + 1);
        semiOrden.pop();
    }
}
```

2.7. Ordenamiento Topologico

Ordena los vertices de un DAG (digrafo aciclico dirigido) de forma que toda arista e = (v->w) cumple que v precede a w en el orden. Sirve para determinar si el grafo no es aciclico.

```
#include <bits/stdc++.h>
#define ll long long
#define forr(i, h) for (ll i = 0; i < h; i++)
#define endl '\n'
using namespace std;

using Digrafo = vector<vector<pair<ll, bool>>>;
vector<ll> ordenTarjan;
vector<bool> visitadoTarjan;
vector<ll> marcasTarjan;
ll ordenadosTarjan;
bool visitarTarjan(Digrafo &g, ll nodo, ll marcaActual) {
    if (visitadoTarjan[nodo]) return true;
    if (marcasTarjan[nodo] == marcaActual) return false;
    marcasTarjan[nodo] = marcaActual;
    bool hayCiclo = false;
    for (auto [vecino, salida] : g[nodo]) {
        if (!salida) continue;
        hayCiclo = hayCiclo && !visitarTarjan(g, vecino, marcaActual);
    }
    visitadoTarjan[nodo] = true;
    ordenTarjan[nodo] = ordenadosTarjan++;
    return !hayCiclo;
}

bool ordenamientoTarjan(Digrafo &g) {
    ordenTarjan = vector<ll>(g.size());
    visitadoTarjan = vector<bool>(g.size(), false);
    marcasTarjan = vector<ll>(g.size(), -1);
    ordenadosTarjan = 0;
    bool esAciclico = true;
    for (ll nodo = 0; nodo < g.size() && esAciclico; nodo++) {
```

```
33          if (visitadoTarjan[nodo]) continue;
34          esAciclico = visitarTarjan(g, nodo, nodo);
35      }
36      return esAciclico;
37  }
```

Teoria de Números

3.1. Minimo Comun Multiplo/Divisor Comun Menor

Calcula el mcm o dcm de dos numeros.

```
#include <bits/stdc++.h>
#define ull unsigned long long
using namespace std;

ull dcmEuclideano(ull n1, ull n2) {
    if (n1 == n2 || n1 == 0 || n2 == 0) return max(n1, n2);
    if (n2 < n1) swap(n1, n2);
    return dcmEuclideano(n1, (n2 % n1));
}

ull mcmEuclideano(ull n1, ull n2) { return (n1 / dcmEuclideano(n1, n2))
   * n2; }
```

3.2. Criba de Eratóstenes

Calcula para todos los numeros menores a un N si son primos y su menor divisor.

```
#include <bits/stdc++.h>
#define ll long long
#define forr(i, h) for (ll i = 0; i < h; i++)
using namespace std;

bool es_primo(int x) {
    for (int i = 2; i < x / 2; i++) {
        if (x % i == 0) {
            return false;
        }
    }
    return true;
}

ll criba(ll MAX, vector<bool> &primos, vector<ll> &minDivisor) {
    primos.clear();
    minDivisor.clear();
    forr(i, MAX + 1) {
        primos.push_back(true);
        minDivisor.push_back(i);
    }

    primos[0] = primos[1] = false;

```

```
    ll cantPrimos = MAX;
    for (int p = 2; p * p <= MAX; p++) {
        if (!primos[p]) continue;
        for (int d = p * p; d <= MAX; d += p) {
            if (primos[d]) {
                primos[d] = false;
                cantPrimos--;
                minDivisor[d] = p;
            }
        }
    }
    return cantPrimos;
}
```

3.3. Exponenciación Binaria

Calcula rapidamente la enecima potencia de un numero. Util para conseguir el inverso modular.

```
#include <bits/stdc++.h>
#define ll long long
using namespace std;
#define MOD ((ll)1e6 + 7)
#define sum(a, b) (((a % MOD) + (b % MOD)) % MOD)
#define res(a, b) (((a % MOD) - (b % MOD) + MOD) % MOD)
#define mul(a, b) (((a % MOD) * (b % MOD)) % MOD)

ll exponenciacionBinaria(ll x, ll e) {
    ll acumulador = 1;
    ll potBinaria = x;
    while (e != 0) {
        if (e & 1) {
            acumulador = mul(acumulador, potBinaria);
        }
        e = e >> 1;
        potBinaria = mul(potBinaria, potBinaria);
    }
    return acumulador;
}

ll inversoModular(ll x) { return exponenciacionBinaria(x, MOD - 2); } // MOD DEBE ser primo
```

Estructuras de Datos

4.1. Segment Tree

Permite realizar cualquier operacion asociativa en rangos.

```cpp
#include <bits/stdc++.h>
#define ll long long
using namespace std;

struct ST {
    // 1. NEUTRO (Declarar SIEMPRE arriba del vector!)
    // - Suma: 0 | Min: 4e18 (o 1e9 en int) | Max: -4e18 | XOR: 0
    ll NEUTRO = 0;
    int n;
    vector<ll> t;

    // 2. OPERACION DE CONSULTA
    // Cambiar segun el problema: a + b, min(a, b), max(a, b), a ^ b
    ll op(ll a, ll b) { return a + b; }

    // 3. CONSTRUCTOR (Usa solo 2 * n de memoria)
    ST(int n) : n(n), t(2 * n, NEUTRO) {}

    // 4. ACTUALIZAR UN PUNTO (Posicion k de 0 a n - 1)
    // - Por defecto ASIGNA/REEMPLAZA el valor: t[k += n] = x
    // - Si el problema pide SUMAR al valor que ya estaba: cambiar '=' por '+='
    void upd(int k, ll x) {
        for (t[k += n] = x; k /= 2;) t[k] = op(t[2 * k], t[2 * k + 1]);
    }

    // 5. CONSULTAR RANGO INCLUSIVO [l, r] (Indices de 0 a n - 1)
    ll query(int l, int r) {
        ll L = NEUTRO, R = NEUTRO;
        for (l += n, r += n; l <= r; l /= 2, r /= 2) {
            if (l & 1) L = op(L, t[l++]);
            if (!(r & 1)) R = op(t[r--], R);
        }
        return op(L, R);
    }
};
```

4.2. Lazy Segment Tree

Permite actualizar rangos rapidamente.

```cpp
#include <bits/stdc++.h>
#define ll long long
using namespace std;

struct LazyST {
    // 1. NEUTROS (Declarar SIEMPRE arriba de los vectores!)
    // - Para SUMA en rango: NEUTRO = 0, LZ_NEUTRO = 0
    // - Para MIN en rango:  NEUTRO = 4e18 (o 1e9 en int), LZ_NEUTRO = 0
    // - Para MAX en rango:  NEUTRO = -4e18, LZ_NEUTRO = 0
    // - Si el update es ASIGNAR (Set) en vez de sumar: LZ_NEUTRO = -1 (o un valor que no exista en el input)
    ll NEUTRO = 0, LZ_NEUTRO = 0;
    int n;
    vector<ll> t, lz;

    // 2. OPERACION DE CONSULTA
    // Cambiar segun lo que pida la query: a + b, min(a, b), max(a, b)
    ll op(ll a, ll b) { return a + b; }

    // 3. CONSTRUCTOR (Tamanno 4 * n)
    // OJO: Si vas a hacer min/max pero cargas los datos iniciales sumando con upd(i, i, x),
    // pone t(4 * n, 0) en vez de t(4 * n, NEUTRO) para que las hojas empiecen en 0 y no en 4e18.
    LazyST(int n) : n(n), t(4 * n, NEUTRO), lz(4 * n, LZ_NEUTRO) {}

    // 4. PROPAGACION (El "post-it")
    void push(int v, int tl, int tr) {
        if (lz[v] == LZ_NEUTRO) return;

        // A) Como afecta el cambio al nodo actual:
        // - Si op es SUMA y update es SUMAR:   t[v] += lz[v] * (tr - tl + 1);
        // - Si op es MIN/MAX y update es SUMAR: t[v] += lz[v];
        // - Si op es SUMA y update es ASIGNAR: t[v] = lz[v] * (tr - tl + 1);
        // - Si op es MIN/MAX y update es ASIGNAR: t[v] = lz[v];
        t[v] += lz[v] * (tr - tl + 1);

        // B) Como se pasa el post-it a los hijos (si no es hoja):
        // - Si el update es SUMAR: usar '+='
        // - Si el update es ASIGNAR (Set): cambiar '+=' por '=' (sobreescribir)
        if (tl != tr) lz[2 * v] += lz[v], lz[2 * v + 1] += lz[v];

        lz[v] = LZ_NEUTRO; // Limpiamos el post-it actual
    }

    // Funciones internas: 'v' es 1-indexado (raiz = 1, hijos = 2*v y 2*v+1)
    // 'tl' y 'tr' son los limites del nodo actual (0 a n-1)
    void upd(int v, int tl, int tr, int l, int r, ll val) {
        push(v, tl, tr);
        if (tl > r || tr < l) return;
        if (tl >= l && tr <= r) {
            // C) Aplicar nuevo post-it:
            // - Si el update es SUMAR:   lz[v] += val;
```

```
            // - Si el update es ASIGNAR: lz[v] = val;
            lz[v] += val;
            push(v, tl, tr);
            return;
        }
        int tm = (tl + tr) / 2;
        upd(2 * v, tl, tm, l, r, val);
        upd(2 * v + 1, tm + 1, tr, l, r, val);
        t[v] = op(t[2 * v], t[2 * v + 1]);
    }

    ll query(int v, int tl, int tr, int l, int r) {
        push(v, tl, tr);
        if (tl > r || tr < l) return NEUTRO;
        if (tl >= l && tr <= r) return t[v];
        int tm = (tl + tr) / 2;
        return op(query(2 * v, tl, tm, l, r), query(2 * v + 1, tm + 1,
            tr, l, r));
    }

    // 5. FUNCIONES PARA USAR EN EL MAIN (Indices l y r van de 0 a n -
        1, inclusivos)
    void upd(int l, int r, ll val) { upd(1, 0, n - 1, l, r, val); }
    ll query(int l, int r) { return query(1, 0, n - 1, l, r); }
};
```

4.3. Trie

Conjunto de strings con acceso e insercion rapida.

```
#include <bits/stdc++.h>
using namespace std;

class Trie {
  public:
    Trie *h[26];
    bool hoja;

    Trie() {
        hoja = false;
        for (int i = 0; i < 26; i++) {
            h[i] = nullptr;
        }
    }

    void insert(string word) {
        Trie *actual = this;

        for (char c : word) {
            int index = c - 'a';
            if (actual->h[index] == nullptr) {
                actual->h[index] = new Trie();
            }
            actual = actual->h[index];
        }
        actual->hoja = true;
```

```
    }

    bool search(string word) {
        Trie *actual = this;

        for (char c : word) {
            int index = c - 'a';
            if (actual->h[index] == nullptr) {
                return false;
            }
            actual = actual->h[index];
        }
        return actual->hoja;
    }

    bool startsWith(string prefix) {
        Trie *actual = this;

        for (char c : prefix) {
            int index = c - 'a';
            if (actual->h[index] == nullptr) {
                return false;
            }
            actual = actual->h[index];
        }
        return true;
    }
};
```

4.4. Union-Find

Implementacion de conjuntos que optimizan la busqueda y la union.

```
#include <bits/stdc++.h>
#define ll long long
#define forr(i, h) for (ll i = 0; i < h; i++)
using namespace std;

vector<ll> predecesoresUF;  // INICIALIZAR
vector<ll> profundidadesUF; // INICIALIZAR

ll initUnionFind(ll elementos) {
    predecesoresUF = vector<ll>(elementos);
    profundidadesUF = vector<ll>(elementos, 0);
    forr(i, elementos) profundidadesUF[i] = i;
}

ll buscarRepresentante(ll i) {
    if (predecesoresUF[i] == i) return i;
    return predecesoresUF[i] = buscarRepresentante(predecesoresUF[i]);
}

void unirConjuntos(ll e1, ll e2) {
    e1 = buscarRepresentante(e1);
    e2 = buscarRepresentante(e2);
    if (e1 == e2) return;
```

```
24      if (profundidadesUF[e1] < profundidadesUF[e2]) {
25          predecesoresUF[e1] = e2;
26      } else if (profundidadesUF[e1] > profundidadesUF[e2]) {
27          predecesoresUF[e2] = e1;
28      } else {
29          predecesoresUF[e1] = e2;
30          profundidadesUF[e1]++;
31      }
32  }
```

Códigos de Ejemplo

5.1. Codigo 1

```cpp
#include <bits/stdc++.h>
#define ll long long
#define forr(i, h) for (int i = 0; i < h; i++)

using namespace std;

class Trie
{
public:
    Trie *h[26];
    bool hoja;

    Trie()
    {
        hoja = false;
        for (int i = 0; i < 26; i++)
        {
            h[i] = nullptr;
        }
    }

    void insert(string word)
    {
        Trie *actual = this;

        for (char c : word)
        {
            int index = c - 'A';
            if (actual->h[index] == nullptr)
            {
                actual->h[index] = new Trie();
            }
            actual = actual->h[index];
        }
        actual->hoja = true;
    }
};

// int cubrecadena(string s, )

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
45
46      string s;
47      cin >> s;
48      int l = s.length();
49      int n;
50      cin >> n;
51
52      vector<string> bloques;
53      string bloque;
54      Trie *trie = new Trie();
55
56      forr(i, n)
57      {
58          cin >> bloque;
59          bloques.push_back(bloque);
60          trie->insert(bloque);
61      }
62
63      int dp[l + 1];
64      dp[l] = 0;
65      for (int i = l - 1; i >= 0; i--)
66      {
67          dp[i] = dp[i + 1];
68          Trie *actual = trie;
69          for (int j = i; j < l; j++)
70          {
71              int indice = s[j] - 'A';
72              if (actual->h[indice] == nullptr)
73              {
74                  break;
75              }
76              actual = actual->h[indice];
77              if (actual->hoja)
78              {
79                  int largo = j - i + 1;
80                  int opcion_actual = largo + dp[i + largo];
81                  if (opcion_actual > dp[i])
82                  {
83                      dp[i] = opcion_actual;
84                  }
85              }
86          }
87      }
88  }
```

5.2. Codigo 2

```
1  #include <bits/stdc++.h>
2  #define ll long long
3  #define forr(i, h) for (int i = 0; i < h; i++)
4  #define ull unsigned long long
5
6  using namespace std;
7
8  const int MAXN = 100005;
```

```
9
10 const ull p1 = 131;
11
12 ull potencias1[MAXN];
13 ull hashes[MAXN + 1];
14
15 void inicializar_potencias()
16 {
17     potencias1[0] = 1;
18     for (int i = 1; i < MAXN; i++)
19     {
20         potencias1[i] = potencias1[i - 1] * p1;
21     }
22 }
23
24 void prefixSum_hashes(const char *s, int *largo)
25 {
26     int n = *largo;
27     for (int i = 0; i < n; i++)
28     {
29         hashes[i + 1] = hashes[i] * p1 + s[i];
30     }
31 }
32
33 ull obtener_hash(ll l, ll r)
34 {
35     ull len = r - l + 1;
36     ull hash = hashes[r + 1] - hashes[l] * potencias1[len];
37     return hash;
38 }
39
40 char s[100005];
41 char a[2005];
42 int main()
43 {
44     inicializar_potencias();
45     scanf("%s", s);
46     int largo_s = strlen(s);
47     prefixSum_hashes(s, &largo_s);
48
49     int n;
50     scanf("%d", &n);
51
52     while (n--)
53     {
54         scanf("%s", a);
55         int largo = strlen(a);
56
57         ull hash_a = 0;
58         for (int i = 0; i < largo; i++)
59         {
60             hash_a = hash_a * p1 + a[i];
61         }
62
63         bool flag = false;
64         for (int i = 0; i <= largo_s - largo; i++) // 100000
65         {
66             ull len = largo;
```

```
            ull hash = hashes[i + largo] - hashes[i] * potencias1[len];
            if (hash_a == hash)
            {
                printf("Y \n");
                flag = true;
                break;
            }
        }
        if (!flag)
            printf("N \n");
    }
}
```

5.3. Codigo 3

```
#include <bits/stdc++.h>
#define ll long long
#define forr(i, h) for (int i = 0; i < h; i++)


using namespace std;

const int MAXN = 100005;

const ll p1 = 131, p2 = 137;
const ll m1 = 1e9 + 7, m2 = 1e9 + 9;

vector<ll> potencias1(MAXN), potencias2(MAXN);
vector<ll> hashes1(MAXN + 1, 0), hashes2(MAXN + 1, 0);

void inicializar_potencias()
{
    potencias1[0] = 1;
    potencias2[0] = 1;
    for (int i = 1; i < MAXN; i++)
    {
        potencias1[i] = (potencias1[i - 1] * p1) % m1;
        potencias2[i] = (potencias2[i - 1] * p2) % m2;
    }
}

void prefixSum_hashes(const string &s)
{
    ll n = s.length();
    for (int i = 0; i < n; i++)
    {
        hashes1[i + 1] = (hashes1[i] * p1 + s[i]) % m1;
        hashes2[i + 1] = (hashes2[i] * p2 + s[i]) % m2;
    }
}

pair<ll, ll> obtener_hash(ll l, ll r)
{
    ll len = r - l + 1;
    ll hash1 = (hashes1[r + 1] - (hashes1[l] * potencias1[len]) % m1) %
       m1;
```

```
41      hash1 = (hash1 + m1) % m1;
42      ll hash2 = (hashes2[r + 1] - (hashes2[l] * potencias2[len]) % m2) %
            m2;
43      hash2 = (hash2 + m2) % m2;
44      return make_pair(hash1, hash2);
45  }
46
47  int main()
48  {
49
50      inicializar_potencias();
51      string s;
52      cin >> s;
53      int largo_s = s.length();
54      prefixSum_hashes(s);
55
56      int n;
57      cin >> n;
58      string a;
59      while (n--)
60      {
61          cin >> a;
62          int largo = a.length();
63
64          ll h1_a = 0, h2_a = 0;
65          for (int i = 0; i < largo; i++)
66          {
67              h1_a = (h1_a * p1 + a[i]) % m1;
68              h2_a = (h2_a * p2 + a[i]) % m2;
69          }
70          pair<ll, ll> hash_a = make_pair(h1_a, h2_a);
71
72          bool flag = false;
73          for (int i = 0; i <= largo_s - largo; i++)
74          {
75              if (hash_a == obtener_hash(i, i + largo - 1))
76              {
77                  cout << "Y" << "\n";
78                  flag = true;
79                  break;
80              }
81          }
82          if (!flag)
83              cout << "N" << "\n";
84      }
85  }
```